Audio Sonification Layer
for the 5D Projected Entity

missvector
github.com/vifirsanova

May 20, 2026

1 Design Principle

The visual system computes 56 nodes moving in $\mathbb{R}^5$, projected to $\mathbb{R}^2$. The audio layer does **not** alter any of that math. It is a read-only observer: one node is designated as the *solo voice*, and three of its computed values are mapped to audio parameters once per frame.

This keeps CPU cost minimal (one sine oscillator) while maintaining a direct, mathematically faithful link between what you see and what you hear.

2 Signal Chain

Oscillator	$\rightarrow$	**Gain Node**	$\rightarrow$	**Master Gain**	$\rightarrow$	**Audio Out**
(frequency f)		(amplitude A)		(mute/solo)		(speakers)

All nodes are instances of `OscillatorNode` and `GainNode` from the Web Audio API. The oscillator type is `'sine'`.

The master gain is used only for muting/unmuting (on/off button). Its target value is either 0 (muted) or 0.18 (active), with a 0.3 s linear ramp to avoid clicks.

3 Mapping: Visual $\rightarrow$ Audio

3.1 Frequency from Screen X

Let i^* be the index of the *red node* (the solo voice). Let $x^* = x_{\text{screen},\, i^*}(t)$ be its projected screen-x coordinate, in the range approximately $[-0.5,\, 0.5]$.

Frequency is set by exponential mapping:

$$\boxed{f(t) = 110 \cdot 2^{\,(x^* + 0.5) \cdot 2.5}} \tag{1}$$

Since $x^* + 0.5 \in [0,\, 1]$, the exponent ranges $[0,\, 2.5]$, giving:

$$f_{\min} = 110 \cdot 2^0 = 110 \text{ Hz}, \qquad f_{\max} = 110 \cdot 2^{2.5} \approx 622 \text{ Hz} \tag{2}$$

The value is clamped to [55, 1200] Hz as a safety margin. Frequency is updated via `linearRampToValueAtTim` with a ramp time of 0.032 s, producing smooth glissandi between frames.

3.2 Amplitude from Depth

Let $z^* = z_{\text{depth},i^*}(t)$ be the projected depth of the red node. Amplitude follows an inverse-distance law:

$$\boxed{A_{\text{depth}}(t) = \frac{1}{1 + 2.5 \cdot |z^*|}} \tag{3}$$

When $z^* = 0$ (node at projection plane): $A_{\text{depth}} = 1$. When $|z^*| = 1$: $A_{\text{depth}} \approx 0.286$. When $|z^*| \to \infty$: $A_{\text{depth}} \to 0$.

This creates a natural sense of spatial proximity: nodes that project "close" to the viewer sound louder.

3.3 Amplitude Modulation from Pulse

The red node's pulse $q_{i^*}(t)$ (same as used for visual breathing) adds tremolo:

$$\boxed{q_{i^*}(t) = 0.6 + \sin(t \cdot 5 + \varphi_{i^*,0}) \cdot 0.4} \tag{4}$$

The pulse is normalized to a modulator:

$$m_{\text{pulse}}(t) = 0.5 + 0.5 \cdot \sin(t \cdot 5 + \varphi_{i^*,0}) \tag{5}$$

3.4 Combined Amplitude

$$\boxed{A(t) = A_{\text{depth}}(t) \cdot \big(0.06 + m_{\text{pulse}}(t) \cdot 0.1\big)} \tag{6}$$

Expanded:

$$A(t) = \frac{0.06 + 0.05 + 0.05 \cdot \sin(t \cdot 5 + \varphi_{i^*,0})}{1 + 2.5 \cdot |z^*|} \tag{7}$$

The constants 0.06 and 0.1 were chosen empirically so that:

- At maximum depth attenuation ($A_{\text{depth}} \approx 0.29$) and minimum pulse ($m_{\text{pulse}} \approx 0$): $A \approx 0.017$ (barely audible).
- At zero depth ($A_{\text{depth}} = 1$) and maximum pulse ($m_{\text{pulse}} \approx 1$): $A \approx 0.16$ (present but not loud).

Final amplitude is clamped to [0.005, 0.25] and applied via `linearRampToValueAtTime` with the same 0.032 s ramp.

4 Control Interface

4.1 Sound On/Off Button

State	Master Gain Target	Button Text
Off (initial)	0 (AudioContext not created)	"sound: off"
Off (muted)	0	"sound: off"
On	0.18	"sound: on"

First click creates the `AudioContext` (browser autoplay policy). Subsequent clicks toggle between mute and unmute with 0.3 s fades.

4.2 Solo Voice Selection

Action	Effect
Left click on canvas	Select new random $i^* \in \{0, \ldots, 55\}$
Right click on canvas	Cycle speed preset (original behavior)

The red node is rendered with distinct visual treatment (red glow, red core, thicker trails), so the audio source is always visually identified.

5 Latency and Timing

All parameter updates use `linearRampToValueAtTime` with $t_{\text{ramp}} = 0.032$ s. At 60 fps (frame interval ≈ 0.0167 s), the ramp spans roughly 2 frames. This is intentionally shorter than the frame interval to ensure the ramp completes before the next update, avoiding scheduling conflicts.

The ramp time is chosen as:

$$t_{\text{ramp}} \approx 2 \cdot \frac{1}{60 \text{ Hz}} \approx 0.033 \text{ s} \tag{8}$$

6 Code Listing

Listing 1: Audio initialization

```
let audioCtx = null;
let audioOn = false;
let audioOsc = null;
let audioGain = null;
let masterGain = null;

function initAudio() {
  if (audioCtx) return;
  audioCtx = new AudioContext();

  masterGain = audioCtx.createGain();
  masterGain.gain.value = 0;
  masterGain.connect(audioCtx.destination);

  audioOsc = audioCtx.createOscillator();
  audioOsc.type = 'sine';

  audioGain = audioCtx.createGain();
  audioGain.gain.value = 0.12;

  audioOsc.connect(audioGain);
  audioGain.connect(masterGain);
  audioOsc.start();

  masterGain.gain.linearRampToValueAtTime(0.18,
    audioCtx.currentTime + 0.3);
  audioOn = true;
}
```

Listing 2: Per-frame audio update

```
if (audioCtx && audioOn && audioOsc) {
  const now = audioCtx.currentTime;
  const rampTime = 0.032;

```

```
  // Frequency from screen X
  const freq = 110 * Math.pow(2,
    (redNodeScreenX + 0.5) * 2.5
  );
  audioOsc.frequency.linearRampToValueAtTime(
    Math.max(55, Math.min(1200, freq)),
    now + rampTime
  );

  // Amplitude from depth + pulse
  const depthAmp = 1.0 / (1.0 + Math.abs(redNodeDepth) * 2.5);
  const redPulse = 0.5 + 0.5
    * Math.sin(t * 5 + phases[redNodeIndex * 5]);
  const amp = depthAmp * (0.06 + redPulse * 0.1);

  audioGain.gain.linearRampToValueAtTime(
    Math.max(0.005, Math.min(0.25, amp)),
    now + rampTime
  );
}
```

Listing 3: On/off toggle

```
audioBtn.addEventListener('click', () => {
  if (!audioCtx) {
    initAudio();
  } else if (audioOn) {
    masterGain.gain.linearRampToValueAtTime(0,
      audioCtx.currentTime + 0.2);
    audioOn = false;
  } else {
    audioCtx.resume().then(() => {
      masterGain.gain.linearRampToValueAtTime(0.18,
        audioCtx.currentTime + 0.3);
      audioOn = true;
    });
  }
});
```

7 Summary of Mappings

Visual Variable	Audio Parameter	Transfer Function
x^*_{screen}	Oscillator frequency	$f = 110 \cdot 2^{(x^*+0.5)\cdot 2.5}$
z^*_{depth}	Amplitude (distance)	$A_{\text{depth}} = \frac{1}{1+2.5\lvert z^*\rvert}$
$q_{i^*}(t)$ (pulse)	Amplitude (tremolo)	$m_{\text{pulse}} = 0.5 + 0.5\sin(t \cdot 5 + \varphi_0)$
—	Combined amplitude	$A = A_{\text{depth}} \cdot (0.06 + m_{\text{pulse}} \cdot 0.1)$